Supplementary Material: ResearchMathAgent
A Self-Evolving Multi-Agent System for Mathematical Proof Research

Anonymous

Abstract

We describe the full system architecture of **ResearchMathAgent (RMA)**, a self-evolving multi-agent platform for automated mathematical proof research. RMA orchestrates a daily *push-forward* cycle in which a *critic agent* discovers proof gaps, *solver agents* attempt repairs, *verifier agents* check correctness, and a structured expert-persona meeting synthesises an action plan. All state, issue history, and research documents are persisted across cycles, enabling compounding improvement over time. This supplementary provides implementation details omitted from the main paper for space reasons.

1 Design Overview

RMA is organised as a stack of seven design levels, each governed by a single guiding insight. The central principle across all levels is *compounding state*: every cycle leaves behind durable, append-only artifacts (issues, JSONL attempt logs, living documents) that become the input context for the next cycle, so the system improves monotonically rather than restarting from scratch. Table 1 summarises the levels and their key insights; the appendices that follow expand each one in detail.

Three cross-cutting invariants hold at every level:

1. **Append-only truth.** Raw history (`strategy_memory.jsonl`, issue threads) is never mutated; all human- and agent-facing documents are derived, regenerable views.
2. **Stable-prefix-first context.** Everything fed to the model is ordered most-stable to least-stable so caching is maximised and re-computation minimised.
3. **Bounded growth.** Deduplication, open-only issue filtering, and fixed retention windows keep context size sub-linear in cycle count.

Level	Key design insight
System (App. A)	One FastAPI orchestrator; datasets fully isolated so issues, documents, and proofs never cross-contaminate.
Pipeline (App. B)	A daily *push-forward* discovers gaps, repairs them, then convenes an expert meeting — idempotent per UTC day.
Agents (App. C)	Four narrow roles (critic, solver, verifier, discussant); the verifier is biased toward rejection to avoid approving wrong proofs.
Issues (App. D)	Agents see *only* open / in-progress issues; resolved issues are hidden to bound context growth.
Meetings (App. E)	Field-matched personas debate over n rounds, then a coordinator synthesises one concrete action plan.
Memory (App. F)	A single append-only JSONL is the source of truth; all living documents are regenerable views over it.
Context (App. H)	A three-block prompt puts stable content first so the prompt cache absorbs 60–70% of input token cost.

Table 1: The seven design levels of RMA and their guiding insights.

A System Architecture

A.1 High-Level Overview

RMA is a client–server web application. The **backend** is a FastAPI server (`webapp/server.py`) that exposes a REST/SSE API and coordinates all agent activity. The **frontend** is a single-page application (`webapp/static/index.html`) providing tabs for Issues, Documents, Push-Forward, Expert Meetings, Export, and System Overview. Agent execution occurs on Google Cloud Vertex AI via AnthropicVertex, with the main process acting as an async orchestrator.

A.2 Dataset Support

RMA is dataset-agnostic at runtime. Two benchmark datasets are currently configured:

Component	Location
API server	`webapp/server.py`
Push-forward engine	`webapp/push_forward.py`
Issue agents	`webapp/issue_agents.py`
Issue tracker	`webapp/issues.py`
Document system	`webapp/rich_documents.py`
Meeting system	`webapp/meet.py`
Dataset store	`webapp/dataset_store.py`
Agent runner	`webapp/agent.py`

Table 2: RMA source module layout.

- **first_proof_1** — 10 problems (q1–q10) spanning stochastic analysis, representation theory, algebraic combinatorics, spectral graph theory, symplectic geometry, and numerical linear algebra.
- **first_proof_2** — 10 additional problems (`prob-01` through `prob-10`) from a second batch of the First Proof benchmark.

Each dataset is fully isolated: issues, documents, and working proofs are stored under separate directory trees and never cross-contaminate.

B Push-Forward Pipeline

The push-forward is the core self-improvement loop, executed once per day (or on demand with `force=True`). The procedure is shown in Listing 1.

Listing 1: Push-forward pseudocode.

```
OPEN = "open,in_progress"

for pid in active_problems:
    # 1. Snapshot open issues
    before = list_issues(pid, status=OPEN)

    # 2. Critic discovers new gaps
    run_discovery_agent(pid)

    # 3. Solver repairs up to N issues
    for iss in list_issues(pid, status=OPEN)[:N]:
        run_resolver_agent(pid, iss.id)

    # 4. Measure delta
    found    = len(after_all) - len(before_all)
    resolved = len(before) - len(after_open)

    # 5. Expert meeting + synthesis
    room = create_room(pid,
                personas=get_personas(pid))
    run_round_offline(room, n_rounds=3)
    run_synthesis(room)   # -> action plan
    save_meeting_notes(pid, room.id)
```

B.1 Tab-Only Issue Policy

A key design invariant is that *agents only ever see open or in-progress issues*. Resolved issues are never included in any agent's context. This is enforced at two levels:

1. **API filter**: `GET /api/issues/{pid}?status=open,in_progress` filters at the storage layer via `list_issues(..., status=...)` in `issues.py`.
2. **Agent system prompts**: All three agent roles (critic, solver, verifier) carry an explicit instruction to query only with `?status=open,in_progress` and to never act on resolved issues.

This prevents agents from reopening closed issues or wasting context tokens on already-solved problems.

C Agent Roles

C.1 Critic Agent (Discovery)

The critic agent reads `problem.tex` and the current working proof (`working_solution.tex`), then: (1) writes a structured mathematical analysis to `analysis.md` covering background, proof structure, gap analysis, difficulty assessment, and suggested approaches; (2) creates one issue per genuine mathematical gap via `POST /api/issues/{pid}`; (3) posts its full analysis as a comment on the primary issue thread; (4) links the analysis document to the issue tracker for UI display. The agent receives up to 50 tool-use turns and 1,200 s of wall-clock time. Prior strategy documents (`strategies.md`, `prefix.md`) and question-level background files are seeded into its workspace to avoid rediscovering known gaps.

C.2 Solver Agent (Resolution)

Each solver agent is assigned a single open issue and works to repair the corresponding gap in `solution.tex`. It reads the full issue thread for prior attempts, proposes and implements a mathematical fix, posts a detailed comment with its reasoning, and transitions the issue to `in_progress` or `resolved` depending on success. On success, the improved proof is saved as the new working proof and a history record is written by `proof_history.py` for audit purposes.

C.3 Verifier Agent (Checking)

The verifier reads an issue thread and the current working proof, then produces a **VERDICT: APPROVED** or **VERDICT: REJECTED** comment. Its system prompt instructs it to err on the side of rejection — a false positive (approving a wrong proof) is considered worse than a false negative.

C.4 Discussion Agent (Peer Review)

A lightweight discussion agent runs round-robin debate on a specific issue thread: critic, solver, verifier, and strategist personas each contribute one response per round, building on the previous participants' comments. This is distinct from the full expert meeting and can be triggered manually from the Issues tab UI.

D Issue Tracking System

D.1 Storage Layout

Issues are stored as JSON files under `webapp/issues/<dataset>/<problem_id>/<issue_id>.json`. Each issue carries: a unique ID, problem ID, title, status (`open` | `in_progress` | `resolved`), type and priority labels, creator identity, and a threaded comment list.

D.2 Issue Taxonomy

Type labels (mutually exclusive): `proof-gap`, `logical-error`, `hallucination`, `unclear-def`, `missing-case`, `wrong-bound`, `scope-issue`, `strategy`, `external-ref`, `notation`.

Priority labels: P0 (Critical) through P3 (Low).

D.3 Caching

A 5-second in-process TTL cache in `list_issues()` avoids repeated filesystem reads during high-frequency API polling (the UI polls every 30 s). The cache is invalidated on any write operation so agents always see fresh state.

E Expert Meeting System

E.1 Persona Assignment

Each mathematical problem is pre-assigned field-appropriate mathematician personas via `get_personas_for_problem()`. For example, problem q4 (finite free probability) receives personas representing a free probabilist, a random matrix theorist, and a combinatorialist.

E.2 Meeting Flow

Each meeting proceeds in four phases:

1. **Room creation**: a coordinator opens a room with a topic and a goal ("agree on highest-priority remaining gaps and produce a concrete action plan").
2. **Multi-round discussion** (n=3 by default): each persona generates one response per round, reading the full transcript to date.
3. **Synthesis**: the coordinator reads the full transcript and produces a structured action plan with a summary and numbered steps.
4. **Documentation**: the transcript and action plan are written to `documents/questions/{pid}/meets/{room_id}-notes.md`.

The action plan summary is persisted in the push-forward run record and surfaced in the System Overview *Meeting Insights* panel.

F Document and Strategy Memory

F.1 Per-Problem Documents

Each problem maintains four living documents under `documents/questions/{pid}/`:

File	Content
`overview.md`	Static reference: problem statement, background, key theorems, why it is hard.
`timeline.md`	Append-only chronological log of every proof attempt.
`progress.md`	Live status: established facts, open gaps, next steps.
`strategies.md`	Strategy space: tried/untried approaches, agent insights.
`prefix.md`	Curated background: theorems, key papers (optional).

Table 3: Per-problem living documents.

Critic agent analyses are automatically appended to `strategies.md` and briefly noted in `progress.md` after each discovery run. Meeting notes are stored in the `meets/` subdirectory.

F.2 System-Level Documents

Design documents not tied to a specific problem reside in `documents/design/`. A companion paper survey (`self-evolving-agents-survey.md`) cataloguing related works lives at the repository root. This supplementary is the primary system design reference and is rendered in the web UI's Design tab.

G Scheduling and Automation

G.1 Daily Push-Forward

The push-forward is gated by a date stamp in `webapp/push_forward_state.json`: if `last_run_date` matches today's UTC date the run is skipped (unless `force=True`), ensuring idempotency even if the trigger fires multiple times. Automated daily execution uses a systemd user timer (crontab is unavailable for the target user):

Listing 2: Systemd timer unit for daily 09:00 push-forward.

```
# ~/.config/systemd/user/rma-push-forward.timer
[Timer]
OnCalendar=*-*-* 09:00:00
Persistent=true   # fires on next boot if machine was off
```

G.2 Run Record Retention

The last 30 push-forward run records are retained in `push_forward_state.json`. Each record stores: date, job ID, problems list, newly-discovered issue titles, resolved issue titles, meeting room ID, action plan summary, and error flags. Older records are pruned automatically to bound file growth.

H Prompt Caching and Context Management

H.1 Token Budgets

Each solver agent call is bounded by fixed token limits:

Parameter	Value	Purpose
`MAX_TOKENS`	32,000	Maximum output tokens per call
`THINKING_BUDGET`	16,000	Extended thinking token budget
`MAX_ITERATIONS`	50	Tool-call iterations per run
Context window	200,000	Claude Opus 4 input limit

Table 4: Token budgets for the RMA solver agent.

H.2 Three-Block Prompt Structure

Each agent run begins with a single user message partitioned into three ordered content blocks, chosen to maximise prompt-cache hit rate:

Listing 3: Three-block first user message (agent.py).

```
Block 0  [cache_control: ephemeral]
  <problem id="q3"> ... problem statement ... </problem>

Block 1  [cache_control: ephemeral]
  <research_context problem="q3">
    ## Research Overview      (overview.tex)
    ## Known Strategies       (strategies.tex)
    ## Proof Progress         (progress.tex)
    ## Work Timeline          (timeline.tex)
    ## Open Issues            (issues/*.json)
  </research_context>

Block 2  [no cache_control]
  "Solve benchmark problem `q3` ..."
```

Block 0 holds the problem statement, which is fixed for the lifetime of the problem and serves as the *primary cache anchor*: after the first call, this block is always a cache hit. Block 1 holds accumulated research context assembled by `build_prefix_context()`: it changes slowly and acts as the *secondary cache anchor*, typically hitting on re-runs within the same day. Block 2 is the solve instruction; it is always placed **last** and never marked cacheable, since Anthropic's prompt caching requires the cached prefix to be a stable prefix of the full input and the trailing instruction may vary.

The system prompt is also marked `cache_control: ephemeral`:

```
system = [{"type": "text",
           "text": SYSTEM_PROMPT,
           "cache_control": {"type": "ephemeral"}}]
```

This yields four potential cache read tokens per call (system + block 0 + block 1 + inter-call message history), reducing input token cost by approximately 60–70% relative to passing the full context uncached on every iteration.

H.3 Research Context Assembly

`build_prefix_context(repo_root, problem_id)` assembles Block 1 from the per-problem document tree:

Source file	Section heading in Block 1
`overview.tex`	`## Research Overview`
`strategies.tex`	`## Known Strategies and Attempts`
`progress.tex`	`## Proof Progress Log`
`timeline.tex`	`## Work Timeline`
`issues/*.json`	`## Open Issues`

Table 5: Inputs assembled into the research context block.

Only open issues (`status` $\in$ {`open`, `in_progress`}) are included. Resolved issues are excluded to bound token growth as the problem matures. Each file prefers the `.tex` version over the legacy `.md` version.

H.4 Document Lifecycle

The document tree feeds forward into future agent calls and backward from completed agent runs:

1. Agent run completes $\Rightarrow$ outcome appended to `strategy_memory.jsonl` (append-only JSONL, never modified).
2. `update_question_document()` reads the full JSONL and rewrites `timeline.tex`, `progress.tex`, and `strategies.tex`.
3. Next agent call reads these files via `build_prefix_context()`.

Because `strategy_memory.jsonl` is the single source of truth, all derived `.tex` files can be fully regenerated at any time by calling

`seed_all_question_documents()` (exposed as `POST /api/docs/regenerate-all`).

H.5 Timeline Deduplication

The timeline `timeline.tex` is derived from `strategy_memory.jsonl` by `_dedup_attempts()`, which collapses *consecutive* identical runs: two attempts are identical if they share the same strategy prefix (first 80 characters), model, outcome, and `issue_count`. Collapsed runs emit a single row annotated with a repetition count $[N\times]$ and the date range [date_start .. date_end]. This prevents the timeline from growing quadratically during stuck phases (e.g. Q3 had 33 consecutive Metropolis–Hastings attempts that collapsed to one row).

A sentinel score `issue_count` $= -1$ indicates the run was aborted (agent stopped or errored) and is mapped to 99 by `_valid_ic()` so it is never mistaken for a near-complete proof (which would be `issue_count` $= 0$ or 1).

H.6 Caching Layer Summary

Five caching layers are active or planned in RMA, ordered by implementation priority:

1. **Anthropic prompt cache** (implemented) — stable prefix ordered first with `cache_control: ephemeral`; saves ≈60–70% of input token cost per call.
2. **In-process issue list TTL** (implemented) — 5 s LRU dict; invalidated on any write; eliminates redundant filesystem reads during high-frequency UI polling.
3. **Document bundle PDF cache** (implemented) — full bundle PDF pre-built at server startup and cached in `documents/pdf/bundle.pdf`; filtered per-question bundles compiled on demand.
4. **Agent result cache** (planned) — SHA-256 keyed JSON files in `webapp/agent_cache/`, 24 h TTL; invalidated on issue or proof changes; saves the full Vertex AI agent cost on unchanged inputs.
5. **HTTP response cache** (planned) — `max-age=30` on document endpoints; `max-age=86400` on static assets.

Metric	Value
Backend (`server.py`)	≈2,300 lines
Frontend (`index.html`)	≈5,400 lines
Issue agents (`issue_agents.py`)	≈600 lines
Push-forward (`push_forward.py`)	≈400 lines
REST API endpoints	40+
Supported datasets	2 (extendable)
Problems per dataset	10
Max agent turns per call	50
Max wall time per agent	1,200 s
Push-forward run retention	30 records
Meeting rounds (default)	3
Daily schedule	09:00 UTC via systemd

Table 6: RMA implementation statistics.

I Implementation Statistics

J Claude Code vs. Claude API vs. Vertex AI

This section clarifies the three distinct execution channels used in RMA and their relationship to each other.

J.1 Billing and Authentication Separation

Although the same model name (e.g. `claude-opus-4-8`) appears in both Claude Code and the webapp's Vertex AI calls, they are *completely separate billing channels*:

- **Claude Code subscription** — billed to Anthropic via your Max/Pro plan; traffic goes to `api.anthropic.com`; authenticated with your Anthropic account.
- **Vertex AI Claude** — billed to your Google Cloud account; traffic goes through `us-east5-aiplatform.googleapis.com`; authenticated via Google ADC (`gcloud auth application-default`); subject to Google Cloud quotas and per-token GCP pricing.

A 429 quota error on Vertex AI has zero effect on Claude Code continuing to function. The two channels are invoice-separated, quota-separated, and auth-separated even when the underlying model weights are identical.

J.2 Agentic Capability by Execution Mode

The webapp uses *two different modes* of model invocation, summarised in Table 7.

The push-forward pipeline uses *pure single-shot text completion* with no tools, no self-correction, and no ability to read files or verify its own output. It calls the most capable model in the least capable mode.

	Claude Code	Solver (`agent`)	Push-forward
API	Direct	Vertex	Vertex
Tools	Yes	Yes	**No**
Multi-turn	Yes	Yes	**No**
Ext. thinking	Yes	Yes	**No**
Streaming	Yes	Yes	**No**

Table 7: Capability comparison across RMA execution modes. "Direct" = Anthropic API; "Vertex" = AnthropicVertex.

J.3 How Agentic Tool Use Actually Works

Claude Code's tools (`Read`, `Bash`, `Edit`, `Write`, etc.) are **not part of the API itself** — they are client-side code. The API is stateless by design:

Listing 4: Stateless API request-response cycle.

```
You -> [prompt + tool definitions] -> API
                                    -> [text or tool_call
    ↪ ]
You execute tool locally
You -> [result] -> API -> [next response]
```

The multi-turn loop, tool execution, and state management all live in the application code (`webapp/agent.py`), not in the API.

J.4 Options for Agentic Behavior

Three approaches are available for adding agentic capability to RMA components:

1. **Build the loop in application code** — define tools, call the API, execute tool calls, feed results back, repeat. This is what `agent.py` already does for the proof solver agents.
2. **Claude Code SDK** — Anthropic's SDK for building Claude Code-style agents; handles loop boilerplate while allowing custom tool definitions.
3. **Extended thinking only** — add `thinking={"type": "enabled", "budget_tokens": N}` to existing `vertex_llm.complete()` calls for deeper reasoning without multi-turn overhead:

Listing 5: Enabling extended thinking on Vertex AI.

```
msg = client.messages.create(
    model="claude-opus-4-8",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[{"role": "user", "content": prompt}],
)
for block in msg.content:
    if block.type == "thinking":
        log("REASONING:", block.thinking)
    elif block.type == "text":
        log("ANSWER:", block.text)
```

J.5 Implication for RMA Push-Forward Quality

The current push-forward cycle runs `claude-opus-4-8` with no thinking, no tools, and no self-correction — analogous to hiring a world-class mathematician and allowing them to respond only via postcard with no scratch paper. Enabling extended thinking on the issue discovery and resolution agents would provide both higher-quality outputs and an inspectable reasoning trace to diagnose systematic failure modes (e.g. the pipeline consistently reporting 9 issues per run regardless of proof state).